Regular-Recursion Obstructions for the Five Cascade Schedules

Hans Brende GPT-6 Astra (OpenAI)

November 2026

Abstract

The five implemented cascade schedules accompanying *Oblivious Linear-Time Tape Transposers Need Macroscopic Crossings* satisfy the regular-recursion obstruction of *Local Coding Barriers and the Limits of Tape Geometry for Matrix Transposition*, Theorem 7.5. We verify its hypotheses on the complete published schedules, including preparation, positioning, recursive stages and output. For each fixed work alphabet, every sufficiently large dyadic instance has controller-bit cost plus correctness deficit at least a positive constant times its input length. The conclusion allows arbitrary nonlinear updates at every visit and holds both for bit entries and for canonical logarithmic entries. The proof is asymptotic; it gives no numerical exclusion threshold. The imported obstruction is Lean-verified. The canonical-width C5 exclusion is also formalized, including its complete stage clock, arbitrary local-rule compiler, regularity and real-deficit bound. Its declarative Lean schedule is matched to the Rust source by a written audit and finite trace comparisons. The other four schedule applications, the bit-to-word simulation and the entry-information extension have written proofs.

Contributions. Hans Brende directed the research. GPT-6 Astra (OpenAI) developed the applications, formalization and text. This memo applies an existing obstruction to the exact cascade sources; it does not claim a new general transposition lower bound.

Contents

1 Result and scope **2**

2 The existing obstruction and the reusable schedule test **3**

3 Verification for the five complete schedules **5**

4 Native compilation, bit entries and the quantitative bound **9**

5 What this settles, and how to screen variants **13**

A Source provenance, formal status and reproduction **14**

B Index of new Lean declarations **15**

1 Result and scope

Paper 16's five cascade schedules pass several of its finite necessary-condition tests. That does not establish compatibility with earlier necessary conditions. Here the relevant earlier result is Paper 2, Theorem 7.5 [3]: a contracting recursion with regular fixed-stage motion cannot perform near-full-entropy transposition with sublinear finite-control bits. The application requires the complete trace, not just the stage that appears most complicated.

Throughout, $d = 2^m$ is the matrix dimension, b the entry width and $L = d^2 b$ the number of input bits. The two width regimes in our conclusion are $b = 1$ and $b = \max(1, \lceil \log_2 d \rceil) = m$. All logarithms in information quantities are base two. The five constructors are exactly those of the published Rust crate at the revision recorded in Appendix A; their names are given in Table 1. For asymptotic statements, their loops and coordinate formulas are read with exact integer arithmetic. The Rust executable realizes finite representable instances; fixed-width integer overflow is not part of the mathematical schedule family.

A *completion* fixes the input-head moves, work-head moves, append times and total duration to those of the constructor. Work tapes start blank, all heads start at zero, and the output starts empty. At each tick, an arbitrary deterministic local rule reads the old controller, raw input symbol and all old scanned work symbols, then updates the controller, writes the work symbols, and emits a bit when prescribed. Rules may depend on d and the public tick index. They may rewrite at *every* visit, including positioning steps marked `PassThrough` in the crate. This is the unrestricted local-update interpretation of the schedule. It includes completions that respect the crate's more restrictive active-write annotations.

Fix a finite work alphabet Γ, of cardinality A, containing a blank symbol. A completion has a nonempty finite controller set Q; write $q = \lceil \log_2 |Q| \rceil$. The initial controller value is public. If the output equals raw row-major transpose on a nonempty set $S \subseteq \{0,1\}^L$, write

$$D = L - \log_2 |S|.$$

No assumption is made about correctness outside S, linearity of the rules, independence of stored symbols, or uniform generation of the rules.

Theorem 1.1 (C: exclusion of all five exact schedules)**.** *For every fixed finite work alphabet there exist $\delta > 0$ and d_0 such that, for each of the five constructors, every dyadic $d \geq d_0$, each of the two width regimes $b = 1$ and $b = \max(1, \lceil \log_2 d \rceil)$, and every completion and nonempty correctness set S as above,*

$$q + L - \log_2 |S| \geq \delta L. \tag{1.1}$$

The constants can be chosen simultaneously for all five constructors and both widths. They are not evaluated numerically here.

Lean (partial): `CascadeObstruction.C5.uniform_c5_gap` [`CascadeObstruction/C5Exclusion.lean`]
Not formalized: The canonical-width C5 case is formalized for the declarative schedule in Lean. Its Rust correspondence, the other four schedules, all bit-width cases and the finite minimum across cases remain written proofs.

The complete written proof is assembled in Section 4, using the source proofs in Section 3 and the cited Lean-verified obstruction in Section 2. For C5 at canonical width, the Lean theorem discharges the compiler, clock and regularity obligations; its proof is outlined in Section 4.4. The verification boundary is recorded in Appendix A.

Full correctness gives $q \geq \delta L$. If $q = o(L)$, the ambient success fraction is at most $2^{-(\delta - o(1))L}$. Thus fixed-state machines are covered, but fixing the number of states is unnecessary. The number

of work tapes and alphabet are fixed; for these constructors there are two or three work tapes. The complete schedule is oblivious even though its data updates may be nonlinear and depend on previous data.

The statement is asymptotic along dyadic dimensions. Neither the smallest legal constructor size nor a tested experimental size is an exclusion threshold. The geometric estimates below hold for every positive integer b, but the transposition conclusion is proved only in the two stated width regimes. It does not decide unrestricted linear-time transposition, question (T) of the series [1].

2 The existing obstruction and the reusable schedule test

We state the hypotheses needed from Paper 2 explicitly. Its native machines have a read-only input, finitely many initially blank work tapes with unit-speed heads, and append-only unreadable output. A tick reads the complete old tuple before writing at the old positions and moving the heads. Head trajectories are frozen after halting.

2.1 Native resource and correctness assumptions

Let $d_i \to \infty$, put $b_i = \max(1, \lceil \log_2 d_i \rceil)$ and $L_i = d_i^2 b_i$. Machines may vary with i. Their number k of work tapes and cell charge $G = \mathrm{cellBits}(M_i) \geq 1$ are fixed. Their state counts satisfy $\lceil \log_2 |Q_i| \rceil / L_i \to 0$. For correct sets W_i there are integers $e_i \geq 0$ with

$$2^{L_i} \leq 2^{e_i} |W_i|, \qquad e_i / L_i \longrightarrow 0. \tag{2.1}$$

Eventually every $w \in W_i$ is a correctly encoded matrix on which the machine halts with its exact transpose. When a time bound is assumed, it is cL_i for one fixed c. There is no correctness premise outside W_i. A real deficit can be rounded up to obtain e_i.

2.2 Charged stages and their actual limiting profiles

A *contracting stage clock* is Paper 2's Definition 7.2. Fix branching B, stages per node J, and constants $0 \leq \rho < 1$, $a \geq 0$, C. At each word w over B symbols, a nonnegative volume $V(w)$ satisfies

$$V(\varnothing) \leq CL, \qquad \sum_{c<B} V(wc) \leq \rho V(w).$$

Each stage (w, j) owns an integer interval $[A(w, j), E(w, j))$ of ticks. These intervals partition *all* ticks exactly, and

$$\sum_{j<J} (E(w, j) - A(w, j)) \leq aV(w).$$

Empty stages are allowed. The stages owned by one node need not be adjacent; in particular a forward stage and a backward stage may surround descendant work. The clock's volume is an accounting quantity, not a free random-access memory bank.

For one fixed node and stage across a sequence, write V_i, A_i, ℓ_i for its volume, actual start and duration, and h_i for one head path. Paper 2's Definition 7.4 requires that every subsequence with $V_i \to \infty$ and $\ell_i > 0$ have a further subsequence on which

$$\frac{h_i(A_i + \min\{\lfloor V_i s \rfloor, \ell_i\}) - h_i(A_i)}{V_i} \longrightarrow v(s) \quad \text{uniformly on } [0, a], \tag{2.2}$$

where v has a finite affine interval cover. The number of pieces may depend on the head and the fixed stage. It need not be bounded across all depths. Bounded-volume or empty occurrences impose no diverging-stage requirement. The profile freezes at its *actual* end.

For flagged clocks, good stages obey this requirement and other stages are exceptional. The family-level hypothesis quantifies over every sufficiently late dimension subsequence, every choice of correct inputs on it and their actual halting times: one can choose clocks and flags, with fixed parameters, for which the good-stage regularity holds and $E_{\mathrm{bad}}/L_i \le \eta + \varepsilon_i$, where $\varepsilon_i \to 0$ is fixed for the family. The clocks may depend on that execution sequence.

Theorem 2.1 (Paper 2, Theorem 7.5)**.** *Under the native resource and correctness assumptions above, suppose every correct input eventually admits an actual halting execution with a contracting stage clock, and the family has flagged clocks with the stated regularity and exceptional allowance* (η, ε_i)*. Then*

$$\frac{1}{2G} \le k\eta.$$

In particular, all-regular clocks with zero exceptional work are impossible. If linear running time is already assumed, the first conclusion holds without separately postulating clock existence for each correct execution.

Cited: [3, Theorem 7.5]; Lean-verified in `TransposeFollowup.RegularMotion.native_flagged_recursion_budget`, `native_fast_flagged_recursion_budget` and `no_native_regular_recursion`. The latter two shortened names use the same namespace.

The proof uses Paper 2's Theorems 6.2 and 6.3. For suitable physical head limits on $[0, c + 1/G]$, every choice of open locally affine regions R_h satisfies

$$\sum_{h<k} \lambda([0, c + 1/G] \setminus R_h) \ge \frac{1}{2G}. \tag{2.3}$$

An all-regular contracting clock supplies locally affine regions whose complements have measure zero, contradicting this inequality. The obstruction is to locally affine motion outside a null set, not merely to a fixed finite number of affine pieces.

2.3 The test used here

It suffices to verify the following facts about a complete size-indexed schedule family:

1. Every tick belongs to a charged stage of a fixed contracting clock. Include all preparation, positioning, wraps, rewinds and output.
2. At each fixed stage, every work head has the regularity (2.2). A growing global number of turns is allowed.
3. The alphabet and tape count are fixed, and the time-dependent local gates can be compiled into native transitions with only $o(L)$ additional controller bits.

These are schedule and model conditions. They do not depend on which nonlinear local functions are chosen. Merely having a contracting clock is insufficient: any linear-time run can be put in one root stage, but that stage need not have a finite-affine limiting profile.

For our unary clocks, $J = 2$, $V_r = L/2^r$, $\rho = 1/2$ and $C = 1$. The following specialization also allows any volumes satisfying the clock's inequalities, without requiring equality in the contraction.

Lemma 2.2 (Dyadic clock accounting)**.** *Any contracting stage clock with $B = 1$, $J = 2$, $\rho = 1/2$, $C = 1$ and $a \geq 0$ satisfies*

$$T \leq 2aL, \qquad \#\{\textit{ticks at depths } r \geq h\} \leq 2a\,2^{-h}L. \tag{2.4}$$

Lean: `CascadeObstruction.DyadicClock.total_time_le`, `CascadeObstruction.DyadicClock.deep_time_le` [`CascadeObstruction/ClockBudget.lean`]

Outline of the formal proof. Apply Paper 2's geometric clock sums with contraction $1/2$, then use the root bound $V(\varnothing) \leq L$ and $1/(1 - 1/2) = 2$. □

After keeping finitely many depths, the remaining physical duration can be made arbitrarily small. This is why finitely many affine pieces *at each fixed depth* suffice.

3 Verification for the five complete schedules

The C1–C4 applications below have written source proofs; C5's clock and regularity are also formalized for its declarative Lean schedule. No application is inferred from numerical tests. We use the pinned `schedules.rs` constructors and `model.rs` movement convention. A positioning call takes the maximum absolute displacement of the participating heads; a head that arrives early stays at its target. Every tick is charged, including ticks whose annotated data operation is a no-op.

Set

$$K = m - 1, \quad M_r = L/2^r \ (1 \leq r \leq K), \quad M_K = 2db \geq 8, \quad \sum_{r=1}^{K} M_r = L - 2db.$$

The root owns preparation and final positioning/output. Depth r owns the complete forward block F_r and backward block B_r, including their entry moves and internal wraps. Deeper stages are empty. These intervals partition the actual timeline. Take $V_r = L/2^r$ at every depth, even after the last nonempty stage. Within a depth-r stage, abbreviate $M = M_r$.

Constructor	tapes	legal dyadic d	$\lvert F_r\rvert + \lvert B_r\rvert$	a
`c1_torus_cascade`	2	≥ 16	$\leq 128M_r$	128
`c2_bfly_cascade`	2	≥ 4	$\leq 128M_r$	128
`c3_ten_entry_cascade`	3	≥ 4	$\leq 80M_r$	80
`c4_no_overwrite_cascade`	3	≥ 4	$\leq 128M_r$	128
`c5_comb_fanout`	2	≥ 4	$\leq 18M_r$	18

Table 1: Safe all-tick budgets. Root work is also at most aL. All geometric bounds hold for integer $b \geq 1$. The legal sizes are *not* numerical exclusion thresholds.

3.1 The common variable-speed loop

The forward loops in C1–C4 use counters c_0, c_1, with cap $M - 1$. Put $u = \lfloor M/3 \rfloor$. At local pre-step time t, their exact positions relative to the starting cells are

$$c_0(t) = \min(t, M - 1), \qquad c_1(t) = \min\big(M - 1, \max(\lceil t/2 \rceil, t - u + 1)\big). \tag{3.1}$$

For C1–C3 the loop has $M + u - 2$ transitions and this formula includes its final configuration. C4 adds one terminal stationary event, giving $M + u - 1$ transitions.

Here is a direct verification of the source recurrence. Maintain $c_1 \le c_0$. While $c_0 \le M-2$, write $M = 3u+v$, $0 \le v \le 2$. Then $c_0 \le 3u$ and

$$\lfloor 2c_0/3 \rfloor + u - c_0 = u - \lceil c_0/3 \rceil \ge 0.$$

The source's condition for advancing head 0 is therefore true whenever it is unfinished. Before c_1 reaches u, head 1 advances precisely when $c_0 \ge 2c_1$; its successive values are $\lceil t/2 \rceil$. At $t = 2u-1$ it reaches u, then advances every tick until capped, giving $t-u+1$. The caps preserve $c_1 \le c_0$, and some unfinished head always moves, so the fallback is unused. Both caps are first reached at $t = M+u-2$. This proves the formula and endpoint counts.

After scaling time and space by M, the frozen profiles converge uniformly, with error $O(1/M)$, to

$$x(s) = \min(s,1), \qquad y(s) = \min(1, \max(s/2, s-1/3)). \tag{3.2}$$

The breaks are $2/3, 1, 4/3$. Monotonicity alone would not establish this finite-affine limit; the recurrence does.

3.2 C1 and C2: preparation and the shared two-tape core

C1's growing row loop. One torus-shift helper first rotates columns within rows. Put $w = db$, $\ell_1 = (d-v)b$, $\ell_2 = vb$ for $0 < v < d$. A nonfinal row costs

$$\ell_1 + (w-1) + \ell_2 + (w+1) = 3w :$$

two scans, the within-row rewind, and the positioning for the next row. The final row costs $2w-1$. The whole phase therefore costs exactly $3L - w - 1$. At the nonfinal row-boundary times $3iw$, both heads have advanced iw relative to their initial positions. Between boundaries, their distances from the speed-$1/3$ profile are at most $2w+2$ and $w+2$, respectively; these bounds also cover the frozen final extension to $3L$. Since $w/L = 1/d$, both scaled paths converge uniformly to $s \mapsto \min(s/3, 1)$.

The following temporary-bank copy is a straight L-tick scan. The row rotation has two straight portions of total length L and a rewind of length $L-1$. Its entry moves are paid. For $v = 0$, the column rotation is instead one straight scan. Each helper has a finite-affine limit whenever its row-rotation fraction converges. For dyadic $d \ge 256$ the four actual parameter pairs (u, v) are

$$(3d/16+1, 16),\ (d-16, 13d/16-1),\ (7d/16+2, 22),\ (d-22, 9d/16-2).$$

Their fractions u/d converge. The near-full rotations retain their macroscopic rewinds; a small complementary segment does not make the whole helper negligible.

Helper $j \in \{1,2,3,4\}$ uses temporary bases $j(L+4)$. Its three large positionings cost at most $3(j+1)(L+4)$; its loops cost at most $6L$. All four helpers therefore cost at most $66L+168$. Loading, seed positioning and seed copy bring preparation to at most $69L + 173 \le 70L$. The shared core starts at work positions $(5(L+4)+L-1, L-1)$, with frame base $5(L+4)$. The eventual parameter formula $d \ge 256$ is used for convergence only; it is not an exclusion cutoff.

C2's fixed prelude. After loading and full reversal, C2 has two shifted forward/reverse pairs with shifts

$$\Delta_{\mathrm{M}} = (d/2)(d-1)b, \qquad \Delta_{\mathrm{L}} = (d-1)b.$$

Their ratios to L tend to $1/2$ and 0. Each pair has two scans of length $L - \Delta$. In particular, the two small-shift positionings vanish after scaling, but their two scans each still have limiting duration 1. Loading, reversal and the four scans cost at most $6L$; their five positionings at most $5L$; seed positioning and copy at most $2L+4$. Thus preparation costs at most $14L$ and has finitely many affine limiting pieces. Core entry is $(L+4+L-1, L-1)$, with frame base $L+4$.

The complete shared stages. At depth r, the second tape allocates

$$Z_r = \mathrm{scr}_r + M_r + 2, \quad \mathrm{comb}_r = \mathrm{scr}_r + 2M_r + 4, \quad \mathrm{scr}_{r+1} = \mathrm{scr}_r + 3M_r + 6.$$

Allocating a name takes no tick, but all moves to these addresses do. In F_r, tape 0 stays in a range $3M + 3$. Tape 1's range is at most $4M + 6$ except at the first depth, where the C1 prelude requires the safe bound $10M + 22$ (C2 is smaller). There are at most five positioning calls and at most $5M$ loop events. Hence

$$|F_r| \le 55M + 110.$$

In B_r, tape 0's range is at most $3M + 3$. At a nondeepest entry, tape 1 is at $\mathrm{scr}_r + 4M + 9 + \mathrm{shift}_{r+1}$, with $1 \le \mathrm{shift}_{r+1} < M/2$; its subsequent positions are between Z_r and $\mathrm{comb}_r + M - 1$. A range $4M + 10$ covers the whole stage, also at the deepest entry. At most six positionings and $8M$ loop events give

$$|B_r| \le 32M + 60, \qquad |F_r| + |B_r| \le 87M + 170 \le 128M.$$

Here $M \ge 8$. The shared routine's variable named `comb` is a straight M-cell copy, not a growing tooth loop.

At core exit tape 1 is at frame_base $+ L + \mathrm{shift}_1 + 3$; tape 0 is at the last cell of its final source. The final positioning and output cost at most $9L$ for C1 and $5L$ for C2. Root work is thus at most $79L$ and $19L$, below $128L$ in both cases.

For each fixed r, the source shifts $\mathrm{shift}_r = (\lfloor M_r/3 \rfloor + b) \bmod M_r$ have ratio $\mathrm{shift}_r/M_r \to 1/3$, since $b/M_r = 2^r/d^2 \to 0$. Normalized entry offsets and durations converge. All ordinary copies, reversals and positionings have finite-affine limits; doubling has at most one cell of two-tick raster error from its affine profiles. Equation (3.2) handles the remaining loop. Finite concatenation, including freezing at the stage end, proves (2.2) at every fixed depth and at the root.

3.3 C3: bounded-entry geometry still has regular stages

For F_r, write A_0 for the current tape-0 base, H for the scratch base and Z for the tape-2 frame. The full coordinate ranges are

$$[A_0, A_0 + 3M - 1], \quad [H - 2M - 5, H + M - 1], \quad [Z, Z + M].$$

The wider tape-1 range includes its incoming position at every depth. At most six positionings cost $4M + 6$ each. The variable loop, the two shifted-copy pieces, their possible joining event, and copy-back cost at most $4M + 1$. Thus $|F_r| \le 28M + 37 \le 40M$.

For B_r, write A_0 for the current tape-0 base and H for the comb base. The full ranges are

$$[A_0, A_0 + 4M + 1], \quad [H - M - 5, H + M - 1], \quad [Z, Z + M].$$

There are 2^r teeth, each of length $M/2^r = d^2 b/4^r \ge 4b$. The comb has $3M - 2 \cdot 2^r - 1$ ticks: its last tooth has no final join. It returns the three heads to (A_0, H, Z). Doubling and the two reversed pieces cost $4M$ and return tape 0 to its new base $A_0 + 2M + 2$. At most five positionings therefore give $|B_r| \le 5(4M + 6) + 7M \le 40M$. The last shifted reverse can leave tape 1 at a nonzero comb offset; the next stage's range above includes it.

Preparation costs $5L$: initial tape-2 positioning, load, tape-0 positioning, reverse copy, and tape-2 return. The loading seed has speed $1/2$ during the first third and then speed 1, up to bounded raster error. At final core exit, tape 1 is at most $4L + 6K$; tape 0 is at its final base and tape 2 at zero. Since $K \le L/6$, positioning and output cost at most $6L$. Root work is at most $11L$. The clock uses $a = 80$. At fixed r, its 2^r teeth are finite, every other loop was covered above, and allocation offsets are geometric sums plus $O(K)$ endpoint terms. Division by L makes the latter vanish. This proves regularity of every fixed stage and both root stages.

3.4 C4: disjoint zones, including all transits

Let X, Y be the fresh tape-0 and tape-1 allocation cursors at a forward stage, and Z its tape-2 frame. The source base is $X - 2M - 2$. The complete stage ranges have lengths at most $4M + 3$, $4M + 6$ and $3M + 4$. At most seven positionings, and at most $6M$ active events, give

$$|F_r| \le 34M + 42.$$

In B_r, the tape-0 and tape-1 ranges have lengths at most $6M + 5$ and $6M + 8$. Tape 2 starts at Z at the deepest stage and at $Z + (7/2)M + 6$ otherwise; its entire range is covered by $6M + 8$ too. There are at most eight positionings. Its comb has exactly 2^r teeth of length $M/2^r$, with $3M - 2 \cdot 2^r$ ticks in total. Initial combine, both doublings, reverse, merge and copy-back together with the comb cost at most $14M$. Therefore

$$|B_r| \le 62M + 64, \qquad |F_r| + |B_r| \le 96M + 106 \le 128M.$$

These bounds include cyclic wraps, the moving side heads and their parked tails. No preservation annotation is used.

Put $S_M = L - 2db$. The seed base is $3S_M + 4K + 2$; complete preparation has duration $9L - 12db + 8K + 6 \le 10L$. Before the final positioning, tape 0 is already at the final base, tape 1 is at $7S_M + 10K - 3$, and tape 2 is at $L/2 + 2$. Positioning plus the $3L$-tick output sweep costs $10L - 14db + 10K - 3 \le 10L$. Root work is at most $20L$.

At fixed depth, the comb has a fixed finite number 2^r of teeth. Each tooth has three monotone legs. Positionings are finite unit/stay pieces, ordinary loops are affine, doubling has bounded raster error, and the variable loop satisfies (3.2). All normalized allocation offsets and shifts converge: they are geometric sums, $\text{shift}_r/M_r \to 1/3$, and endpoint errors $O(K)/L \to 0$. The seed load and reverse copy have finitely affine limits, as does the final sweep (tapes 0 and 1 have speed 1/3; tape 2 has speed 1 then parks). Thus all actual fixed stages are regular. The unbounded *total* number of comb teeth does not violate the theorem's local fixed-stage hypothesis.

3.5 C5: comb fanout

Loading costs L and leaves the work heads at $(L - 1, 0)$. Assign the following $L - 1$-tick reset to root preparation. This changes only clock ownership. Every forward and backward recursive stage now starts and ends at $(0, 0)$; subsequent initial resets take no ticks. Root preparation costs $2L - 1$, and the final output sweep costs L.

Write $H = M/2$, $N = 2^r$, and $h = M/N$. Expanding the literal loops and positioning calls into constant-move blocks and two-tick repeated blocks gives

$$|F_r| = 11H - 2, \qquad |B_r| = 20H - 2N - 2.$$

The backward count includes all N comb teeth, their last stationary event, both reverse half-loops and the final reset. Thus $|F_r| + |B_r| \le 18M$, and root work $3L - 1$ is also at most $18L$. The forward stage uses 13 blocks; the backward stage uses $4N + 10$. These counts, endpoints and the complete clock partition are proved in Lean. Their identification with the Rust loops is a source audit, not a formal semantics for Rust.

For each fixed depth, replace each repeated block by the straight path with the same average velocity and duration. A block has period one or two, so its sampled path differs from this ramp by at most two cells. At most $J_r = 4 \cdot 2^r + 13$ blocks suffice for either stage, including zero-duration padding. The normalized error is at most $2J_r/V_r$, which tends to zero whenever that stage's

charged volume diverges. Normalized block starts and durations lie in $[0, 18]$ and slopes in $[-1, 1]$. Compactness of this finite parameter space supplies a uniform subsequential limit that is a finite sum of affine ramps. Consequently every actual fixed stage is regular, for every sequence of positive entry widths. The unbounded number of stages is paid by the geometric tail.

3.6 From the bounds to the clock hypothesis

The stage ranges above follow inductively from the allocation bases and the actual last position of each preceding loop. They include the deepest stage and the first forward entry; these are the endpoints at which an unpaid reset would otherwise be easy to introduce. With the root budgets, Table 1 proves a complete clock and (2.4).

For finite concatenation of stage profiles, convergent normalized endpoints and durations suffice: all head paths are unit-Lipschitz, so perturbing a junction time by ϵ changes a position by at most ϵ. Each raster error tends to zero, and a fixed stage has only finitely many pieces. The frozen terminal piece is included. Thus the fixed-stage regularity requirement holds on every diverging sequence of admitted dimensions and on every further subsequence. Since the schedules do not depend on the input, this supplies the complete correct-input quantifiers of Theorem 2.1, with all stages good and no exceptional work.

4 Native compilation, bit entries and the quantitative bound

This section assembles Theorem 1.1. The native compiler and the canonical-width C5 assembly are formalized; the bit-to-word bridge below has a written proof. The bit-to-word construction is the parallel oblivious simulation already described by Harvey and van der Hoeven [4, Section 7.6]. We check its motion and correctness-set accounting because a time bound alone would not transfer the affine-motion obstruction.

4.1 Time-dependent gates cost only logarithmically many clock bits

The exact compiler is formalized independently of C5.
Lean: `CascadeObstruction.Compiler.run_denote`, `CascadeObstruction.Compiler.stepN_eq`, `CascadeObstruction.Compiler.haltsAt_iff`, `CascadeObstruction.Compiler.state_count`, `CascadeObstruction.Compiler.state_bits_le` [`CascadeObstruction/CompilerCore.lean`]

Outline of the formal proof. For a completion of duration T, form a native finite controller with states (t, s), $0 \le t \le T$, $s \in Q$. At $t < T$, use the prescribed local rule, movements and append event, then increment t. At $t = T$, halt. This has at most $(T + 1)|Q|$ states and hence at most

$$q + \lceil \log_2(T + 1) \rceil$$

controller bits. Each source tick is one actual native transition; the old tuple is read before any of its work symbols is overwritten. The input, work tapes, output and their positions agree step by step. There is no oracle for time and no free access. The native transition table may depend on d, exactly as allowed by Paper 2's nonuniform theorem. Since $T = O(L)$, the added bit cost is $O(\log L) = o(L)$. □

Prescribed head moves and append times are preserved on every input, with arbitrary simultaneous local data updates.

Lean: `CascadeObstruction.Compiler.scheduled_work_trajectory`,
`CascadeObstruction.Compiler.scheduled_input_trajectory`,
`CascadeObstruction.Compiler.scheduled_append`,
`CascadeObstruction.Compiler.scheduled_oblivious`
[`CascadeObstruction/CompilerSchedule.lean`]

Outline of the formal proof. Induct on the public tick index. Each input and work position changes by the prescribed unit move, independently of the data rule. The append decision is the prescribed Boolean. After tick T, the native machine halts and its trajectories freeze. □

At canonical width, suppose along some $d_i \to \infty$ both $q_i/L_i \to 0$ and $D_i/L_i \to 0$. Compile these completions, set $e_i = \lceil D_i \rceil$, and apply Theorem 2.1 to the all-regular clocks of Section 3. Its conclusion would be $1/(2G) \le 0$, a contradiction. Padding a smaller alphabet into one fixed alphabet, if necessary, fixes its cell charge G.

4.2 An oblivious bit-to-word lift preserving the limiting motion

Let $N = d^2$. Consider a bit-entry completion taking $T = O(N)$ ticks, with k fixed work tapes, alphabet Γ, controller Q, and correctness set $S \subseteq \{0,1\}^N$. Put $b = \log_2 d$ and $L = bN$. The canonical word input already consists of interleaved bit planes: coordinate p of plane ℓ is at raw address $bp + \ell$. Replace every old work cell p by its block of b cells $bp, \dots, bp + b - 1$, on the same work tape over the same alphabet. Keep b old controllers in the new finite control, plus a public clock.

An old tick is simulated in exactly $3b$ native ticks. Initially every head is at the left endpoint bp of its old cell's block. In the first b ticks, visit lanes $\ell = 0, \dots, b-1$ from left to right. Read lane ℓ's controller, its raw bit and all its old work symbols, apply the old local rule, and update that lane only. Emit its bit if this old tick is an append event. Advance all heads after the first $b-1$ lane ticks and stay after the last. Every old read tuple is obtained before that lane's simultaneous writes.

Every head is now at $bp + b - 1$. If its old prescribed move was $\Delta \in \{-1, 0, 1\}$, its next target is $b(p + \Delta)$, at distance at most $2b - 1$. Use the next $2b$ ticks to reach it, parking when it arrives. These transit ticks preserve every scanned work symbol and all lane controllers and emit nothing. Thus all heads reach the next block's left endpoint together. Intermediate reads grant no additional data access: they are ignored except for preserving the symbol read. Negative and right-exterior old raw cells expand to entirely exterior blocks. Induction proves exact simulation of every lane.

The resulting native program has unchanged alphabet and tape count, time $3bT$, and at most

$$(3bT + 1)|Q|^b \quad \text{states}, \qquad bq + \lceil \log_2(3bT + 1) \rceil \quad \text{controller bits}.$$

Each old output coordinate emits all b lanes consecutively in their correct order. The lift is correct on the interleavings of the product set S^b; there are $|S|^b$ of them. Their deficit is $b(N - \log_2 |S|)$. No preliminary bit-plane rearrangement is used.

Let h be one original trajectory, linearly interpolated between ticks, and h' its lifted trajectory. Within the block for old tick t, h' stays within $2b$ of $bh(t)$. The original interpolation changes by at most one cell over that tick. Thus, with frozen continuations,

$$\left| \frac{h'(\tau)}{bN} - \frac{h(\tau/(3b))}{N} \right| \le \frac{3}{N} \qquad (\tau \ge 0). \tag{4.1}$$

If the original physical paths converge to $f(s)$, the lifted physical paths converge to $f(s/3)$. A locally affine limit outside a null set retains that property under this dilation.

The geometric clock proof in Section 3 holds at $b = 1$. For any finite retained depth, its stage endpoints and profiles converge. The deep middle interval has duration tending to zero uniformly as the retained depth increases, by (2.4). Unit-Lipschitz continuity controls the displacement it can contribute. Consequently the complete original paths have locally affine limits outside their countable piece endpoints and the recursive accumulation time. The frozen halt tail is constant. Equation (4.1) gives the same property to the lifted limits. If the original bit completions had $q + D = o(N)$, their lifts would satisfy the canonical resource and near-full correctness assumptions. Paper 2's native affine-time bound (2.3) is then contradicted. The lifted trace need *not* equal the canonical-width cascade constructor. It only needs the already established limiting-motion property. Oblivious head and append schedules are essential to this simulation.

4.3 A positive uniform deficit

Lemma 4.1 (Sequential uniform-gap criterion)**.** *For each natural size n, let $\mathcal{A}_n$ be any type of admissible objects and let $c_n : \mathcal{A}_n \to [0, \infty)$ be a cost. Suppose that for every sequence $n_i \to \infty$ and $x_i \in \mathcal{A}_{n_i}$, the costs $c_{n_i}(x_i)$ do not converge to zero. Then there are $\delta > 0$ and N such that $c_n(x) \geq \delta$ for all $n \geq N$ and all $x \in \mathcal{A}_n$.*

Lean: `CascadeObstruction.exists_uniform_positive_gap` [`CascadeObstruction/UniformGap.lean`]

Outline of the formal proof. Failure supplies $n_i \geq i$ and x_i with $0 \leq c_{n_i}(x_i) < 1/(i + 1)$. The sizes diverge and the costs tend to zero by squeezing. No compactness of the object types is required. □

Corollary 4.2 (Native regular-clock classes)**.** *Consider a size-indexed class of native machines with canonical input length $L(n)$, fixed work-tape count and cell charge, and fixed $a \geq 0$. Each instance supplies a correct-input set W, an integer deficit e with $2^{L(n)} \leq 2^e|W|$, and a complete unary two-stage clock with parameters $(\rho, C) = (1/2, 1)$ for each of its correct halting executions. Suppose every diverging sequence chosen from the class has the full family-level regularity of Section 2. Then, for one $\delta > 0$ and all sufficiently large n, every instance in the class satisfies*

$$\frac{\lceil \log_2 Q_{\text{native}} \rceil + e}{L(n)} \geq \delta,$$

where Q_{native} is its number of controller states.

Lean: `CascadeObstruction.no_vanishing_native_dyadic_cost`, `CascadeObstruction.uniform_native_dyadic_gap` [`CascadeObstruction/NativeObstruction.lean`]

Outline of the formal proof. Vanishing combined cost forces both nonnegative summands to vanish. Paper 2's `no_native_regular_recursion` rules out that sequence. Apply Lemma 4.1. In Lean, the machine has `TM.q + 1` states and the bit count is `Nat.clog 2 (TM.q+1)`. The integer deficit and clock witnesses are fields of `CascadeObstruction.NativeDyadicInstance`; regularity remains an explicit hypothesis on the class. "Dyadic" here describes the clock contraction, not a restriction on the native matrix dimension. □

Proof (not formalized). To finish the proof of Theorem 1.1, fix an alphabet, one constructor and one of the two widths. The preceding arguments rule out every diverging sequence of completions with $(q + D)/L \to 0$. If no positive uniform gap held eventually, one could choose strictly increasing legal dyadic dimensions d_j and completions with

$$0 \leq \frac{q_j + D_j}{L_j} < \frac{1}{j + 1}.$$

Both nonnegative summands would tend to zero. Rounding the deficit and adding the public clock preserve this convergence, giving the forbidden sequence. Hence some positive gap and finite threshold exist for this constructor and width. Take the minimum of the ten positive gaps and the maximum of their thresholds. This proves Eq. (1.1) simultaneously. The sequential-to-uniform step is also checked in the new Lean library; the remaining schedule and simulation premises are supplied by the written proofs. For C5 at canonical width, all these premises are also discharged in Lean below. □

This argument does not estimate the gap or threshold. Finite experiments cannot turn its existential constants into certified numerical ones.

4.4 The complete canonical-width C5 formalization

The C5 part of Theorem 1.1, at canonical width, is checked as a theorem about a specified finite tick word, not about a supplied regularity certificate.

Lean: `CascadeObstruction.C5.uniform_c5_gap`, `CascadeObstruction.C5.completion_family_regular` [`CascadeObstruction/C5Exclusion.lean`]

Outline of the formal proof. The type `CascadeObstruction.C5.CorrectCompletion` contains a legal dimension $d = 2^{K+1}$ with $K \geq 1$, a finite controller, arbitrary local data rules attached to the C5 tick word, and a nonempty set of correctly transposed inputs. It contains no clock, approximation or regularity hypothesis.

The tick word concatenates root preparation, forward stages $1, \ldots, K$, backward stages $K, \ldots, 1$, and root output. Distinct stage labels and prefix sums construct a complete partition of the actual tick indices. The exact bounds in Section 3 give the clock with $a = 18$. Extracting each contiguous stage from the full word identifies its frozen displacement with the block sequence used in the ramp argument. The resulting regularity theorem applies to all positive widths and all eligible subsequences of actual stages.

The compiler then supplies a native machine with the same work alphabet, two work tapes, the same symbols and output, and precisely the prescribed head paths. Its additional controller cost is at most $\lceil \log_2(36L + 1) \rceil$, uniformly over the local rules. For $D = L - \log_2 |S|$, rounding to $e = \lceil D \rceil$ gives $2^L \leq 2^e |S|$ and $e < D + 1$. Apply the native uniform-gap theorem and absorb the clock cost and this one-bit rounding error into a smaller positive gap. This proves the real-deficit conclusion with all model and schedule obligations discharged. □

The reusable clock, regularity and deficit-transfer interfaces are recorded by the following declarations.

Lean: `CascadeObstruction.C5.clock`, `CascadeObstruction.C5.trace_time_bound` [`CascadeObstruction/C5Clock.lean`]

Lean: `CascadeObstruction.C5.clock_regular` [`CascadeObstruction/C5Regularity.lean`]

Lean: `CascadeObstruction.Raster.curve_subsequence`, `CascadeObstruction.Raster.regular_of_ramp_approximation` [`CascadeObstruction/RasterRegularity.lean`]

Lean: `CascadeObstruction.uniform_native_real_deficit_gap` [`CascadeObstruction/GapTransfer.lean`]

Outline of the formal proof. The clock declaration packages the proved partition and budgets. The regularity proof combines finite-product compactness for ramp parameters with a vanishing-error estimate. The deficit-transfer theorem rounds the correctness deficit internally and removes any uniformly sublinear compiler overhead. These interfaces reuse Paper 2's obstruction. □

4.5 Public initial words and paid entry information

Proposition 4.3 (Complete entry information)**.** *The same five-schedule bound holds, with possibly smaller $\delta > 0$, if arbitrary public initial work words are allowed. More generally, let $Z(X)$ specify the entire initial work contents at all visited cells and the initial controller, with the same zero starting head positions, empty output and one common set of later rules. Under the uniform law on a nonempty correctness set S,*

$$q + L - \log_2 |S| + H(Z) \geq \delta L. \tag{4.2}$$

Not formalized; a complete proof is given below.

Proof (not formalized). The complete schedule visits only an $O(L)$ coordinate range on each tape. A constant number of outward and return scans installs any public initial word and restores all starting heads. Hardwire the public symbols in the nonuniform transition table, ignore the raw input during these scans, and install the prescribed initial controller. The time is $O(L)$, the additional clock has $O(\log L)$ bits, and the extra profiles are finitely affine. The preceding obstruction and sequential argument apply uniformly over all public words. The bit case uses the same lane lift.

For input-dependent entry information, fix each nonempty fibre S_z. Its complete entry is now public, so $\log_2 |S_z| \leq (1 - \delta)L + q$ with the same constants. Averaging under uniform $X \in S$ gives $H(X \mid Z) \leq (1-\delta)L+q$. As Z is a deterministic function of X, $\log_2 |S| = H(X) = H(X \mid Z)+H(Z)$, proving (4.2). □

Thus public setup or sublinear joint entry information cannot rescue the same core. A precomputed transpose supplies L bits of entry information and is not excluded by this statement. It still needs a paid producer. An output prefix, input-dependent starting heads, or an input-dependent choice of later rules is not part of this interface; none may be omitted from the information bill by calling it preparation.

5 What this settles, and how to screen variants

The exact five implemented families in Paper 16's Section 10.3 are excluded in the fixed-alphabet native interpretation of its cascade question [2, Problem 11.4]. They may pass the reported finite crossing, separator and skeleton tests: those are necessary conditions, and did not test the older all-regular-recursion obstruction. The proof does not use experimental cut values or literal meanings assigned to intermediate banks.

For a new variant, the reusable obligation is a charged stage decomposition with regular actual fixed-stage profiles. Finite affine preparation and cleanup, bounded-raster changes, or more teeth whose number is finite at each fixed depth do not evade this test when the deep-stage time is still uniformly summable. Nor does allowing arbitrary nonlinear rules or writes at every positioning visit. One must verify the new complete trace; a name such as "cascade" or a picture of its large-scale shape is insufficient. No unlisted gallery variant or historical comparator is classified here.

A potential escape is a stage with a genuinely non-piecewise-affine limit on positive physical time, or a failure of the fixed-stage or tail hypotheses. Such an escape is only from this obstruction, not evidence that the schedule can compute transpose. Likewise, growing work alphabets, other width regimes and adaptive head or append schedules require separate arguments. The imported theorem itself allows adaptive machines under its quantified clock hypotheses; our five source applications and the bit-lane bridge use fixed schedules.

For the impossibility program, this memo prevents further local-gate searches on already excluded complete traces. For the construction program, it identifies the relevant failure mode of these candidates: microscopic irregularities and unbounded recursion depth still leave countably affine macroscopic motion with no exceptional-time allowance. A new construction must both escape that obstruction and perform the ordered computation. The unrestricted question (T) remains open.

A Source provenance, formal status and reproduction

All five source applications are checked against the original published experiments crate at repository revision

`9ffa8f2e5494988731a37c0bed8b0be6a046e5e8`.

The exact source ranges in `experiments/src/schedules.rs` are: shared C1/C2 core 405–559; C1 helper and constructor 562–695; C2 698–779; C3 954–1141; C4 1144–1532; C5 1535–1654. The movement convention is in `experiments/src/model.rs`, 585–715. The manifest in `verification/source-pins.json` gives SHA-256 hashes for the complete Rust code and Cargo files and the principal imported Lean interfaces. It is a targeted provenance manifest, not a replacement for the transitive Lean package pins. C5 additionally has a declarative Lean tick word. Its correspondence with the exact-integer Rust constructor is established by the source audit and the finite comparison below, not by a formal semantics for Rust.

A.1 What Lean verifies

The imported obstruction and clock machinery are verified in Paper 2. The new library verifies Lemmas 2.2 and 4.1 and Corollary 4.2, the arbitrary local-rule compiler, and the complete canonical-width C5 exclusion described in Section 4.4. Its final theorem has no simulation, clock or regularity hypothesis. Those Lean statements are authoritative; the outlines explain them. The declarations depend only on `propext`, `Classical.choice` and `Quot.sound`. The generated declaration index follows this section, and `lean/PaperAxioms.lean` audits every new declaration cited with a formal-status line.

The following remain complete *written* proofs, not Lean proofs:

- the C1–C4 source-to-clock, endpoint and limiting-profile applications;
- the all-instance correspondence between the Rust C5 constructor and its declarative Lean tick word;
- the $3b$ bit-lane simulator, its product correctness set and motion estimate (4.1);
- the remaining cases of Theorem 1.1 and the ten-family finite minimum; real-deficit rounding and compiler-overhead removal are already formalized in the reusable transfer theorem;
- the public-initialization and joint-entry-information extension in Proposition 4.3.

In particular, a successful build verifies the specified Lean C5 word, not the Rust compiler or all five source bindings. The source-specific proofs were separately reviewed against the pinned sources.

Readers need a full repository clone with sibling papers in place. From the memo's `lean/` directory run

```
lake build
lake env lean PaperAxioms.lean
```

Dependency setup follows the second paper's README; the memo reuses its library through a relative path and keeps its Lean and Mathlib pins. From the memo directory, regenerate or check the formal index with

```
python3 tools/lean_index.py
python3 tools/lean_index.py --check
```

Build this document from `paper/` with `latexmk -pdf -interaction=nonstopmode main.tex`.

A.2 Finite implementation controls

From the memo directory, run

```
python3 tools/check_source_pins.py
python3 tools/lane_lift_control.py
python3 -B tools/check_geometry.py
python3 tools/check_c5_lean_trace.py
```

The scripts use the Python standard library; the geometry and C5 comparison also invoke the unchanged original Rust crate with `-offline -locked`. The C5 comparison needs the current Lean library built first. Do not disable Python assertions with `-O`. Optional output flags and receipt-comparison commands are documented in `tools/README.md`. The saved receipts are deterministic and use no machine-specific paths.

The lane control compares independent ordinary-machine and lifted-machine interpreters on 4,928 inputs, executing 569,088 physical transitions. It checks controllers, full work contents, positions, outputs and motion error, and rejects three deliberately incorrect simulators. It includes a tiny true transpose and a nonlinear two-tape computation; it is not a cascade completion.

The geometry control uses ten original crate exports: each C1–C5 at $d = 16$, $b = 1$ and $b = 4$. It checks 273,065 original steps, append ordinals and targets, safe total-time bounds, 24 variable-loop profiles, and C1's row timing and affine shadow. At $d = 16$, C1's eventual parameter formula for $d \geq 256$ has not stabilized. No finite test establishes the asymptotic threshold, proves convergence, verifies all stage budgets, or supplies a data program that transposes.

The C5 comparison exports the actual Lean tick word and compares every input/work move, append flag and resulting head position with the original Rust export. Eight cases at $d = 4, 8, 16$ and widths among $1, 3, \log_2 d$ check 38,504 ticks; three deliberate mutations are rejected. The receipt pins the Lean source and compiled objects. This is a finite transcription check, not an all-instance equivalence proof. The asymptotic C5 theorem comes from Lean's checked clock, regularity and native-obstruction proof.

B Index of new Lean declarations

Generated with the axiom audit from this memo's formal-status lines.

`CascadeObstruction/C5Clock.lean` `CascadeObstruction.C5.clock`, `CascadeObstruction.C5.trace_time_bound`.

`CascadeObstruction/C5Exclusion.lean` `CascadeObstruction.C5.completion_family_regular`, `CascadeObstruction.C5.uniform_c5_gap`.

`CascadeObstruction/C5Regularity.lean` `CascadeObstruction.C5.clock_regular`.

CascadeObstruction/ClockBudget.lean CascadeObstruction.DyadicClock.deep_time_le, CascadeObstruction.DyadicClock.total_time_le.

CascadeObstruction/CompilerCore.lean CascadeObstruction.Compiler.haltsAt_iff, CascadeObstruction.Compiler.run_denote, CascadeObstruction.Compiler.state_bits_le, CascadeObstruction.Compiler.state_count, CascadeObstruction.Compiler.stepN_eq.

CascadeObstruction/CompilerSchedule.lean
CascadeObstruction.Compiler.scheduled_append,
CascadeObstruction.Compiler.scheduled_input_trajectory,
CascadeObstruction.Compiler.scheduled_oblivious,
CascadeObstruction.Compiler.scheduled_work_trajectory.

CascadeObstruction/GapTransfer.lean
CascadeObstruction.uniform_native_real_deficit_gap.

CascadeObstruction/NativeObstruction.lean
CascadeObstruction.no_vanishing_native_dyadic_cost,
CascadeObstruction.uniform_native_dyadic_gap.

CascadeObstruction/RasterRegularity.lean CascadeObstruction.Raster.curve_subsequence, CascadeObstruction.Raster.regular_of_ramp_approximation.

CascadeObstruction/UniformGap.lean CascadeObstruction.exists_uniform_positive_gap.

Also named in the text CascadeObstruction.C5.CorrectCompletion, CascadeObstruction.NativeDyadicInstance.

References

[1] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 9ffa8f2e5494988731a37c0bed8b0be6a046e5e8. URL: https://github.com/HansBrende/papers/tree/9ffa8f2e5494988731a37c0bed8b0be6a046e5e8/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[2] Hans Brende, Claude Opus 5.5 (Anthropic), and Gemini 4 Argon (Google DeepMind). Oblivious linear-time tape transposers need macroscopic crossings. Manuscript, Lean formalization and Rust experiments, repository HansBrende/papers, November 2026. Pinned revision 9ffa8f2e5494988731a37c0bed8b0be6a046e5e8; Section 10.3 and Problem 11.4. URL: https://github.com/HansBrende/papers/tree/9ffa8f2e5494988731a37c0bed8b0be6a046e5e8/Oblivious-Linear-Time-Tape-Transposers-Need-Macroscopic-Crossings.

[3] Hans Brende and GPT-6 Astra (OpenAI). Local coding barriers and the limits of tape geometry for matrix transposition. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 9ffa8f2e5494988731a37c0bed8b0be6a046e5e8; Theorems 6.2, 6.3 and 7.5. URL: https://github.com/HansBrende/papers/tree/9ffa8f2e5494988731a37c0bed8b0be6a046e5e8/Local-Coding-Barriers-and-the-Limits-of-Tape-Geometry-for-Matrix-Transposition.

[4] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition, 2025. Section 7.6. URL: https://arxiv.org/abs/2503.22848v1, arXiv:2503.22848v1.